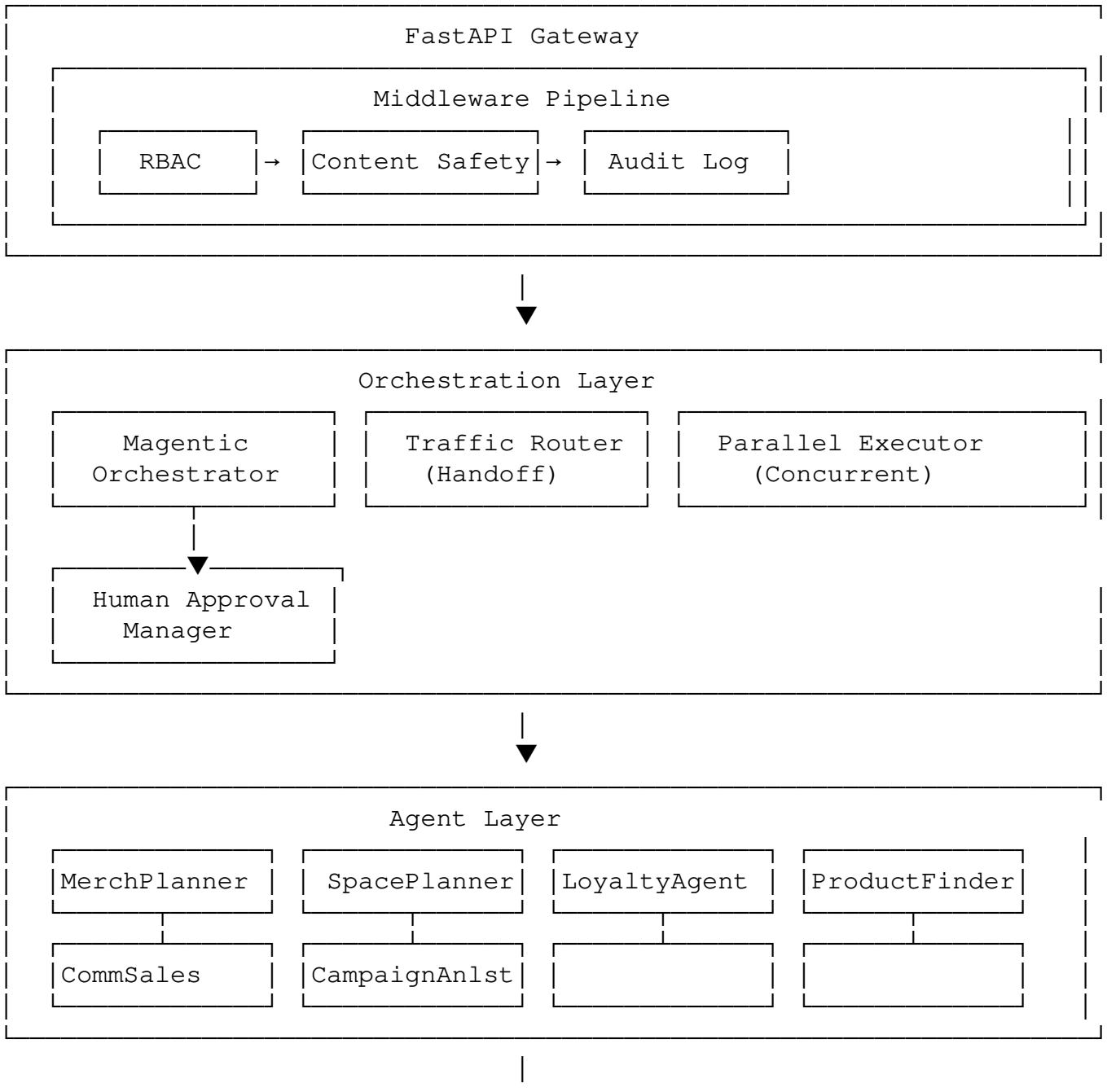
MAFGA Multi-Agent POC

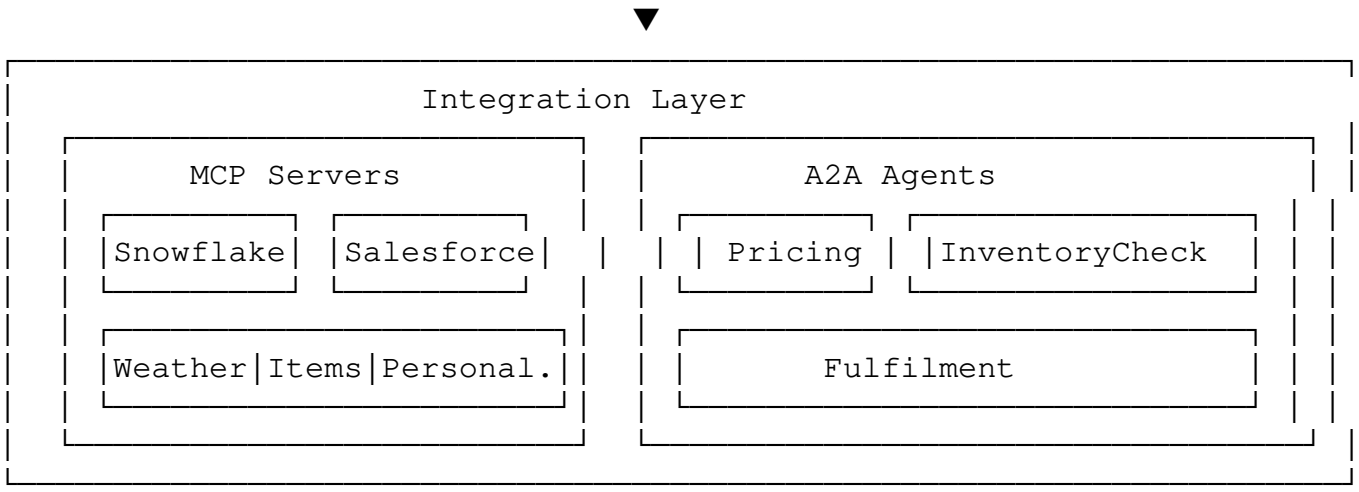
A production-ready Multi-Agent system built using **MAF 1.0 GA** (Microsoft Agent Framework General Availability) architecture patterns.

🏗️ Architecture Overview

This POC implements the Magentic orchestration pattern with:

- **Task-ledger based planning** - Agents coordinate through a shared plan
- **Human-in-the-Loop (HITL)** - Approval gates for critical actions
- **Multi-agent coordination** - Specialized agents for different domains
- **MCP Protocol integration** - Standard tool calling interface
- **A2A Protocol support** - Agent-to-agent communication





Project Structure

```
Multiagent-MAFGA-Arch/
├── api/                      # FastAPI application
│   ├── main.py              # App entry point
│   ├── routes.py            # API route definitions
│   └── models.py            # Pydantic models
├── agents/                   # Domain agents
│   ├── base_agent.py        # Base agent class
│   ├── agent_factory.py     # Cloud-agnostic factory
│   ├── merch_planner.py     # Merchandise planning
│   ├── space_planner.py     # Space planning
│   ├── loyalty_agent.py     # Loyalty & personalization
│   ├── products_finder.py # Product search
│   ├── commercial_sales.py # B2B sales
│   └── campaign_analyst.py # Campaign analysis
├── orchestration/           # Orchestration patterns
│   ├── magentic_orchestrator.py  # Task-ledger planning
│   ├── traffic_router.py         # Intent-based routing
│   ├── parallel_executor.py      # Concurrent execution
│   └── human_approval.py         # HITL approval
├── mcp_servers/             # MCP tool servers (mock)
│   ├── base_mcp_server.py
│   ├── snowflake_mcp.py
│   ├── salesforce_mcp.py
│   ├── weather_mcp.py
│   └── ...
├── a2a_agents/              # A2A protocol support
│   ├── a2a_client.py
│   ├── a2a_server.py
│   └── mock_a2a_agents.py
├── middleware/              # Request middleware
│   ├── rbac_middleware.py
│   ├── content_safety_middleware.py
│   └── audit_log_middleware.py
├── memory/                  # Conversation memory
│   ├── redis_memory.py
│   └── in_memory.py
```

```
├── checkpoint/              # State checkpointing
│   └── file_checkpoint.py
├── skills/                  # Agent skills
│   ├── skill_loader.py
│   └── skill_registry.py
├── observability/           # Telemetry
│   ├── telemetry.py
│   ├── tracing.py
│   └── metrics.py
├── auth/                    # Authentication
│   └── entra_auth.py
├── config/                  # Configuration
│   └── settings.py
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── .env.example
```

🚀 Quick Start

Prerequisites

- Python 3.11 +
- Docker & Docker Compose (optional)
- Azure subscription (for production)

Local Development

1. **Clone and setup**:

   ```
   cd Multiagent-MAFGA-Arch
   python -m venv .venv
   source .venv/bin/activate  # Windows: .venv\Scripts\activate
   pip install -r requirements.txt
   ```

2. **Configure environment**:

   ```
   cp .env.example .env
   # Edit .env with your settings
   ```

3. **Run the API**:

   ```
   python -m api.main
   # Or with uvicorn
   uvicorn api.main:app --reload
   ```

4. **Access the API**:

   - Swagger UI: http://localhost:8000/docs
   - ReDoc: http://localhost:8000/redoc
   - Health: http://localhost:8000/health

Docker

```
# Start all services
docker-compose up -d

# With observability stack
docker-compose --profile observability up -d

# View logs
docker-compose logs -f api
```

🔧 Configuration

Environment Variables

Variable	Description	Default
`AGENT_PROVIDER`	LLM provider (azure_foundry, openai, anthropic, etc.)	azure_foundry
`AZURE_FOUNDRY_PROJECT_NAME`	Azure AI Foundry project name	-
`AZURE_OPENAI_API_KEY`	Azure OpenAI API key	-
`REDIS_HOST`	Redis host for memory	localhost
`ENTRA_TENANT_ID`	Azure EntraID tenant	-
`ENTRA_CLIENT_ID`	Azure EntraID app client ID	-

See .env.example for all options.

LLM Provider Configuration

The system supports multiple LLM providers via the cloud-agnostic factory:

```
# Azure AI Foundry (default)
AGENT_PROVIDER=azure_foundry
AZURE_FOUNDRY_PROJECT_NAME=your-project

# Azure OpenAI
AGENT_PROVIDER=azure_openai
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com

# OpenAI
AGENT_PROVIDER=openai
OPENAI_API_KEY=sk-...

# Anthropic Claude
AGENT_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...

# Local Ollama
AGENT_PROVIDER=ollama
OLLAMA_HOST=http://localhost:11434
```

📚 API Usage

Invoke an Agent

```
curl -X POST http://localhost:8000/agents/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "agent_name": "MerchPlanner",
    "query": "What is the sell-through rate for paint category?"
  }'
```

Run Orchestration

```
curl -X POST http://localhost:8000/orchestration/run \
  -H "Content-Type: application/json" \
  -d '{
    "goal": "Analyze paint category and recommend promotional strategy"
    "orchestration_type": "magentic",
    "require_approval": true
  }'
```

Health Check

```
curl http://localhost:8000/health
```

Testing

```
# Run all tests
pytest

# Run with coverage
pytest --cov=. --cov-report=html

# Run specific module tests
pytest orchestration/tests/
pytest agents/tests/
```

Security

- **EntraID Authentication**: All API endpoints can be protected with Azure EntraID JWT tokens
- **RBAC**: Role-based access control for agent invocations
- **Content Safety**: Automatic PII detection and content filtering
- **Audit Logging**: All operations logged for compliance
- **Secrets Management**: Uses `SecretStr` for sensitive data, never logged

Observability

The system includes comprehensive observability:

- **Tracing**: Distributed tracing with OpenTelemetry
- **Metrics**: Request counts, latencies, error rates
- **Logging**: Structured logging with correlation IDs

Viewing Traces (with Jaeger)

```
docker-compose --profile observability up -d
# Open http://localhost:16686
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes with tests
4. Submit a pull request

License

MIT License - see LICENSE for details.

Acknowledgments

Built following the Microsoft Agent Framework 1.0 GA architecture patterns.